Implementation

Query 1: Switch to Database and Create Collection

Command:

```
test> use purvadb;
purvadb> db.createCollection("users")
```

Output:

```
switched to db purvadb
{ ok: 1 }
```

Query 2: Insert Multiple Documents (insertMany)

Command:

```
purvadb> db.users.insertMany([
  { name: "Ram Kumar",          age: 28, sales: 1200 },
  { name: "Shrinivas Shastry", age: 35, sales: 950  },
  { name: "Kavita Sharma",     age: 40, sales: 1800 },
  { name: "Rohit Sharma",      age: 40, sales: 4800 },
  { name: "Arjun Jadeja",      age: 35, sales: 4000 }
])
```

Output:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69e70541da7a8ffe4444ba89'),
    '1': ObjectId('69e70541da7a8ffe4444ba8a'),
    '2': ObjectId('69e70541da7a8ffe4444ba8b'),
    '3': ObjectId('69e70541da7a8ffe4444ba8c'),
    '4': ObjectId('69e70541da7a8ffe4444ba8d')
  }
}
```

Query 3: Insert a Single Document (insertOne)

Command:

```
purvadb> db.users.insertOne({ name: "Virat Kohli", age: 38, sales: 5100 })
```

Output:

```
{
  acknowledged: true,
  insertedId: ObjectId('69e7055ada7a8ffe4444ba8e')
}
```

Query 4: Find Documents with Condition (age > 30)

Command:

```
purvadb> db.users.find({ age: { $gt: 30 } })   // users older than 30
```

Output:

```
[
  { _id: ObjectId('...ba8a'), name: 'Shrinivas Shastry', age: 35, sales: 950  },
  { _id: ObjectId('...ba8b'), name: 'Kavita Sharma',     age: 40, sales: 1800 },
  { _id: ObjectId('...ba8c'), name: 'Rohit Sharma',      age: 40, sales: 4800 },
  { _id: ObjectId('...ba8d'), name: 'Arjun Jadeja',      age: 35, sales: 4000 },
  { _id: ObjectId('...ba8e'), name: 'Virat Kohli',       age: 38, sales: 5100 }
]
```

Query 5: Aggregation – Total Sales ($sum)

Command:

```
purvadb> db.users.aggregate([
  { $group: { _id: null, totalSales: { $sum: "$sales" } } }
])
```

Output:

```
[ { _id: null, totalSales: 17850 } ]
```

Query 6: Aggregation – Average Age ($avg)

Command:

```
purvadb> db.users.aggregate([
  { $group: { _id: null, averageAge: { $avg: "$age" } } }
])
```

Output:

```
[ { _id: null, averageAge: 36 } ]
```

Query 7: Create Index on Age Field (Ascending)

Command:

```
purvadb> db.users.createIndex({ age: 1 })
```

Output:

```
age_1
```

Query 8: Create Index on Sales Field (Descending)

Command:

```
purvadb> db.users.createIndex({ sales: -1 })
```

Output:

```
sales_-1
```

Query 9: List All Indexes (getIndexes)

Command:

```
purvadb> db.users.getIndexes()
```

Output:

```
[
  { v: 2, key: { _id: 1   }, name: '_id_'     },
  { v: 2, key: { age: 1   }, name: 'age_1'    },
  { v: 2, key: { sales: -1}, name: 'sales_-1' }
]
```

Query 10: Query Performance Analysis (explain with executionStats)

Command:

```
purvadb> db.users.find({ age: { $gt: 30 } }).explain("executionStats")
```

Output (Key Fields):

```
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'purvadb.users',
    winningPlan: {
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { age: 1 },
        indexName: 'age_1',
        direction: 'forward',
        indexBounds: { age: [ '(30, inf.0]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned:         5,
    executionTimeMillis: 0,
    totalKeysExamined: 5,
    totalDocsExamined: 5,
  },
  serverInfo: {
```

```
    host: 'gescoe-OptiPlex-Tower-7020',
    port: 27017,
    version: '7.0.31'
  },
  ok: 1
}
```

Query 11: Aggregation – Maximum Sales ($max)

Command:

```
purvadb> db.users.aggregate([
  { $group: { _id: null, maxSales: { $max: "$sales" } } }
])
```

Output:

```
[ { _id: null, maxSales: 5100 } ]
```

Query 12: Aggregation – Minimum Age ($min)

Command:

```
purvadb> db.users.aggregate([
  { $group: { _id: null, minAge: { $min: "$age" } } }
])
```

Output:

```
[ { _id: null, minAge: 28 } ]
```

Query 13: Aggregation – Count of Documents ($count)

Command:

```
purvadb> db.users.aggregate([
  { $count: "totalUsers" }
])
```

Output:

```
[ { totalUsers: 6 } ]
```